



Live Application — WeatherVault running on EC2 (IP: 13.201.34.225)

EC2	Running	Secrets Manager	Connected	S3 Storage	10 files	Overall Health	Good
Contents							
01	About AWS Secrets Manager						
02	Application Overview						
03	Tech Stack						
04	Architecture Diagram						
05	Implementation — Step by Step						
	5.1 EC2 Instance Setup						
	5.2 IAM Role Configuration						
	5.3 S3 Bucket Creation						
	5.4 Secrets Manager Setup						
06	API Reference						
07	Screenshots & Verification						

01

About AWS Secrets Manager

AWS Secrets Manager is a fully managed service that helps you protect access to your applications, services, and IT resources. It enables you to easily rotate, manage, and retrieve database credentials, API keys, and other secrets throughout their lifecycle — without hardcoding sensitive information in your source code.

Why Use Secrets Manager?

Hardcoding credentials such as API keys directly in application code is a serious security risk. If the code is pushed to a version control system (e.g., GitHub), those credentials become publicly exposed. Secrets Manager solves this by:

- **Centralised storage** — All secrets stored in one secure, auditable location.
- **Fine-grained access control** — IAM policies control exactly which services and users can retrieve each secret.
- **Automatic rotation** — Secrets can be automatically rotated on a schedule without downtime.
- **Encryption at rest** — All secrets are encrypted using AWS KMS (AES-256) by default.
- **Audit trail** — Every access attempt is logged in AWS CloudTrail for compliance.
- **SDK integration** — Applications retrieve secrets at runtime via the AWS SDK — no plaintext files needed.

How It Works

Step	Action	Description
1	Store Secret	Developer creates a secret (key-value pairs or plaintext) in Secrets Manager via Console or CLI.
2	Attach Policy	An IAM policy grants the EC2 instance role permission to call <code>secretsmanager:GetSecretValue</code> .
3	App Retrieves	At runtime, the Flask application calls the AWS SDK to fetch the secret by its ARN or name.
4	Cache in Memory	The application caches the secret values to avoid repeated API calls and reduce latency.
5	Use Securely	The API key is used to call OpenWeatherMap — it never appears in code, logs, or environment files.

■ In this project, a single secret named `weather-dashboard/api-credentials` stores 8 configuration values including the OpenWeatherMap API key, S3 bucket name, region, and application metadata. The EC2 instance accesses it via an attached IAM role — no credentials are stored on the server.

02

Application Overview

WeatherVault is a real-time weather dashboard web application built entirely on AWS. It allows users to search for current weather conditions and forecasts for any city in the world. The application demonstrates how multiple AWS services work together in a production-grade architecture — using EC2 for compute, S3 for storage, and Secrets Manager for secure credential management.

Key Features

- Real-time weather data for any city using the OpenWeatherMap API.
- Quick-access buttons for 7 featured cities (London, New York, Tokyo, Paris, Sydney, Dubai, Mumbai, Toronto).
- 5-step weather forecast view per city.
- Live status indicators showing EC2, Secrets Manager, and S3 health in the UI.
- Weather search history automatically logged to S3.
- API keys and configuration stored securely in AWS Secrets Manager — never in code.
- Dark glassmorphism UI built with vanilla HTML, CSS, and JavaScript.

Project Folder Structure

```
weather-dashboard/  
■■■ setup_secrets.py # Creates the secret in AWS Secrets Manager  
■■■ ec2_userdata.sh # Bootstrap script run on EC2 first launch  
■■■ backend/  
■ ■■■ app.py # Flask REST API – 11 endpoints  
■ ■■■ secrets_manager.py # AWS Secrets Manager client with caching  
■ ■■■ s3_manager.py # S3 operations with caching  
■ ■■■ requirements.txt  
■■■ frontend/  
■ ■■■ index.html # Dark glassmorphism UI  
■ ■■■ style.css  
■ ■■■ app.js  
■■■ s3_setup/  
■■■ setup_s3.py # Creates S3 bucket and uploads config files  
■■■ cities.json # Featured cities data  
■■■ upload_assets.py
```

03

Tech Stack

Layer	Technology	Version / Notes
Backend API	Python Flask	3.0 — REST API with 11 routes
Frontend	HTML + CSS + JS	Vanilla stack, dark glassmorphism design
Web Server	Nginx	Reverse proxy on port 80 → Flask port 5000
Process Manager	Systemd	Keeps Flask running as a background service
Compute	AWS EC2	t2.micro, Ubuntu 22.04 LTS (Free Tier)
Object Storage	AWS S3	Standard class — logs, configs, assets
Secret Storage	AWS Secrets Manager	Encrypted key-value pairs, IAM-controlled access
Identity	AWS IAM	Instance role granting EC2 access to S3 + Secrets Manager
Weather Data	OpenWeatherMap API	Free tier — current weather + 5-step forecast
OS	Ubuntu 22.04 LTS	EC2 hosted, managed via SSH
Language	Python 3.10+	boto3 SDK for all AWS interactions

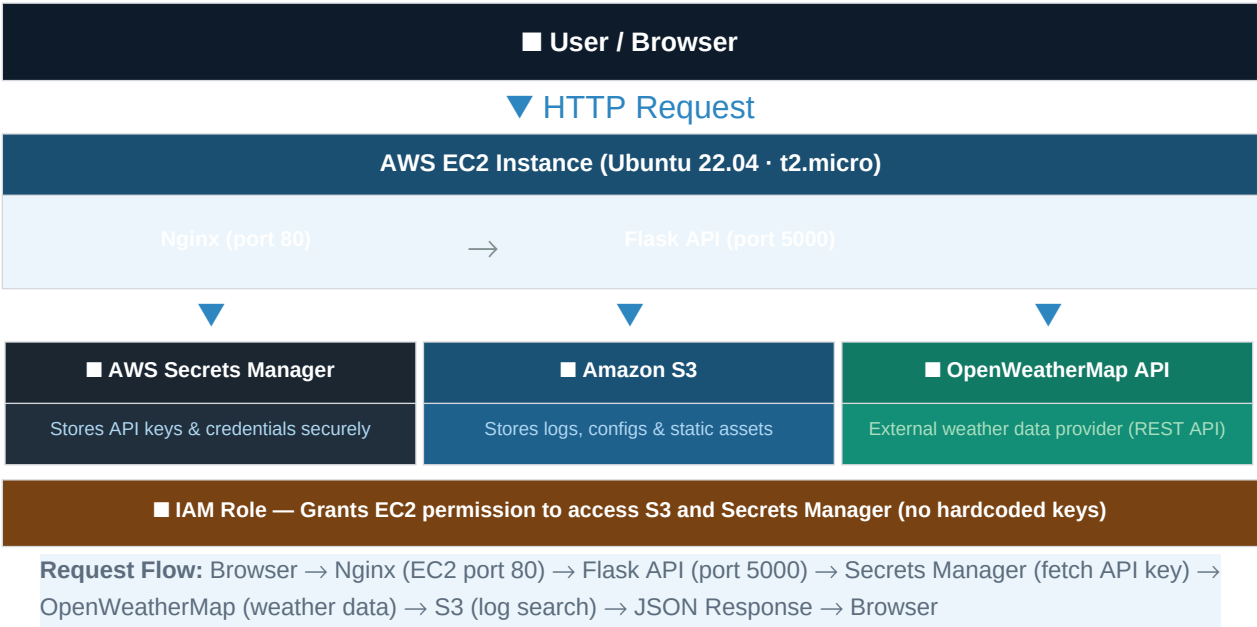
AWS SDK — boto3

The application uses **boto3** (the official AWS SDK for Python) to interact with both Secrets Manager and S3. The EC2 instance carries an IAM role, so boto3 automatically discovers credentials through the instance metadata service — no access keys are stored on the server.

04

Architecture Diagram

The diagram below illustrates the complete request flow from the user's browser through each AWS service to the final weather data response.



05

Implementation — Step by Step

The following steps describe exactly what was done to build and deploy WeatherVault on AWS. Each step is independent and must be completed in order.

5.1 EC2 Instance Setup

An EC2 instance was launched to host the Flask backend and Nginx reverse proxy. EC2 (Elastic Compute Cloud) provides resizable virtual servers in the cloud. The free-tier eligible **t2.micro** instance type was used running **Ubuntu 22.04 LTS**.

Parameter	Value Configured
Instance Name	WeatherVault-Server
AMI	Ubuntu 22.04 LTS (Free tier eligible)
Instance Type	t2.micro (1 vCPU, 1 GB RAM)
Key Pair	New key pair created and .pem file downloaded for SSH access
Security Group	Custom group: SSH (port 22), HTTP (port 80), Flask (port 5000)
IAM Role	Attached after creation — see Section 5.2
User Data	Bootstrap script (ec2_userdata.sh) runs on first boot

Security Group Inbound Rules:

Rule Type	Port	Source	Purpose
SSH	22	My IP only	Secure admin access via terminal
HTTP	80	0.0.0.0/0	Public web access through Nginx
Custom TCP	5000	0.0.0.0/0	Direct Flask API access (testing)

EC2 Bootstrap — ec2_userdata.sh:

```
#!/bin/bash
apt-get update -y && apt-get install -y python3-pip nginx git
git clone https://github.com/YOUR_USERNAME/weather-dashboard.git
cd weather-dashboard/backend && pip3 install -r requirements.txt
# Configure Nginx as reverse proxy to port 5000
# Create systemd service to keep Flask running
```

5.2 IAM Role Configuration

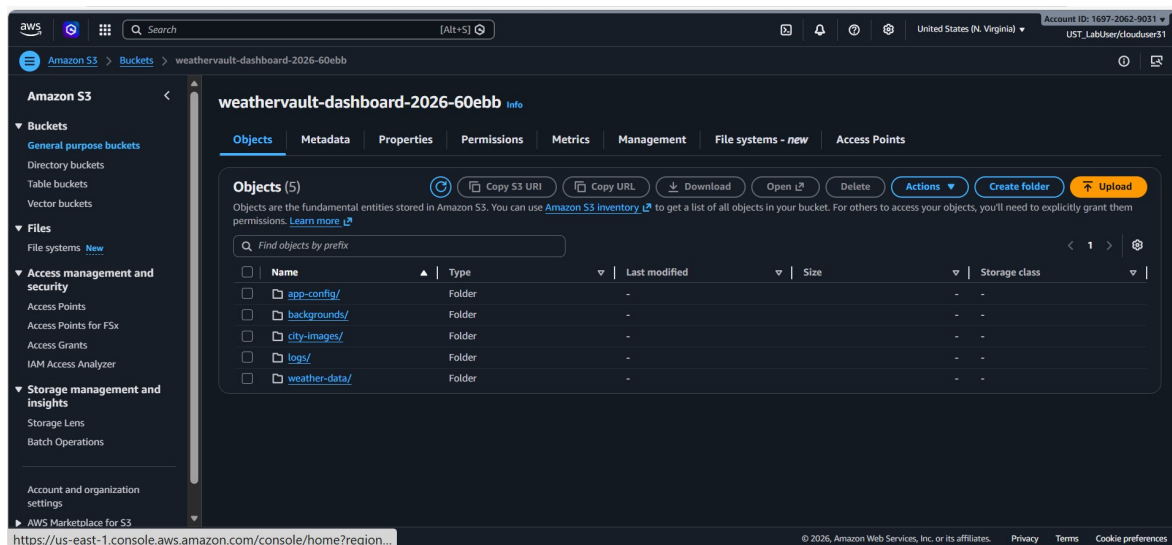
An IAM (Identity and Access Management) role was created and attached to the EC2 instance. An IAM role is an AWS identity with specific permissions. When attached to EC2, the instance can call AWS services without storing any credentials on the server — boto3 automatically uses the role via the instance metadata service (IMDS).

IAM Configuration	Details
Role Name	WeatherVault-EC2-Role
Trusted Entity	EC2 (allows EC2 instances to assume this role)
Policy 1	AmazonS3FullAccess — Read/write access to S3 buckets
Policy 2	SecretsManagerReadWrite — Read secrets from Secrets Manager
Attached To	WeatherVault-Server EC2 instance

■ Because the IAM role is attached at the instance level, no `AWS_ACCESS_KEY_ID` or `AWS_SECRET_ACCESS_KEY` environment variables are needed on the server. This follows AWS security best practices — credentials are never stored in code or config files.

5.3 S3 Bucket Creation

Amazon S3 (Simple Storage Service) was used to store application configuration, weather search logs, background images, and city images. A dedicated bucket was created in the same region as the EC2 instance to minimise latency and data transfer costs.



S3 Bucket — weathervault-dashboard-2026-60ebb showing 5 organised folders

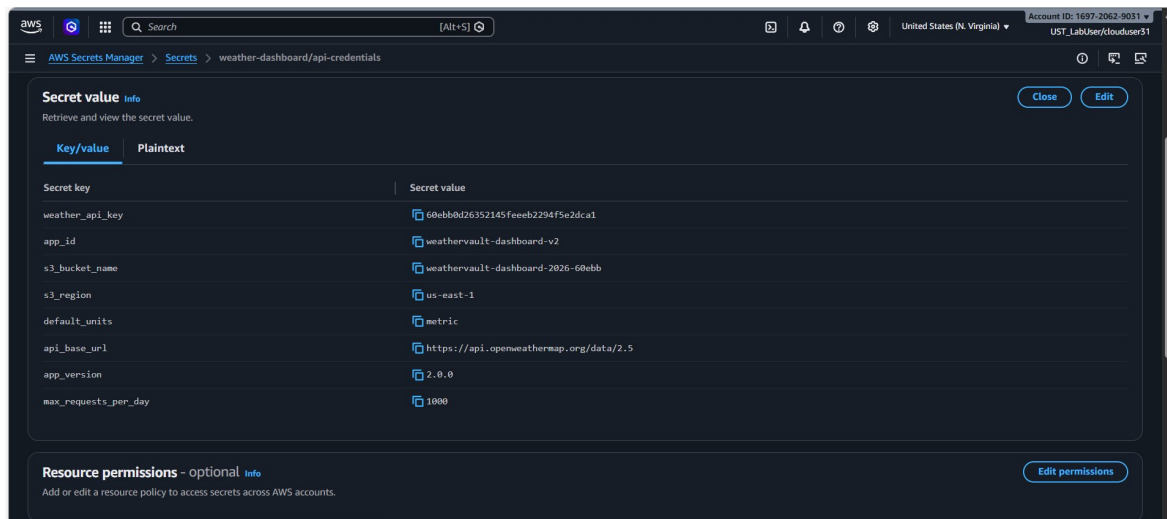
Folder	Contents	Purpose
app-config/	config.json, app-info.json, weather-tips.json	Application settings read at startup
backgrounds/	Background images (.jpg, .webp)	UI background images served via S3
city-images/	City-specific images	Shown on city weather cards
logs/	JSON log files (one per weather search)	Audit trail of all user searches
weather-data/	Cached weather API responses	Reduces external API calls

Bucket setup was done via `setup_s3.py`:

```
import boto3
s3 = boto3.client('s3', region_name='us-east-1')
s3.create_bucket(Bucket='weathervault-dashboard-2026-60ebb')
# Upload config files to app-config/ folder
s3.upload_file('cities.json', BUCKET, 'app-config/cities.json')
```


5.4 Secrets Manager Setup

AWS Secrets Manager was used to store all sensitive credentials and configuration values. A single secret named **weather-dashboard/api-credentials** was created containing 8 key-value pairs. The Flask application retrieves these at startup using the boto3 SDK and caches them in memory.



AWS Secrets Manager — weather-dashboard/api-credentials showing all 8 stored keys

Secret Key	Value	Description
weather_api_key	60ebb0d26352145feeb...	OpenWeatherMap API key
app_id	weathervault-dashboard-v2	Application identifier
s3_bucket_name	weathervault-dashboard-2026-60ebb	S3 bucket name
s3_region	us-east-1	AWS region for S3
default_units	metric	Temperature unit (°C)
api_base_url	https://api.openweathermap.org/data/2.5	OpenWeatherMap base URL
app_version	2.0.0	Current app version
max_requests_per_day	1000	Daily API call limit

Secret was created via `setup_secrets.py`:

```
import boto3, json
client = boto3.client('secretsmanager', region_name='us-east-1')
secret = {
    'weather_api_key': 'YOUR_OPENWEATHERMAP_KEY',
    's3_bucket_name': 'weathervault-dashboard-2026-60ebb',
    's3_region': 'us-east-1', ...
}
client.create_secret(
    Name='weather-dashboard/api-credentials',
```

```
SecretString=json.dumps(secret)
)
```

How Flask retrieves the secret at runtime (secrets_manager.py):

```
import boto3, json
client = boto3.client('secretsmanager', region_name='us-east-1')
response = client.get_secret_value(
    SecretId='weather-dashboard/api-credentials'
)
secrets = json.loads(response['SecretString'])
API_KEY = secrets['weather_api_key'] # Used for OpenWeatherMap calls
```

■ The EC2 IAM role provides the boto3 client with temporary credentials automatically. The secret is fetched once on application startup and cached. This means the OpenWeatherMap API key is never written to disk, never in environment variables, and never in source code.

06

API Reference

The Flask backend exposes 11 REST endpoints. All routes are prefixed with **/api** and return JSON responses. The Nginx reverse proxy forwards all requests from port 80 to Flask on port 5000.

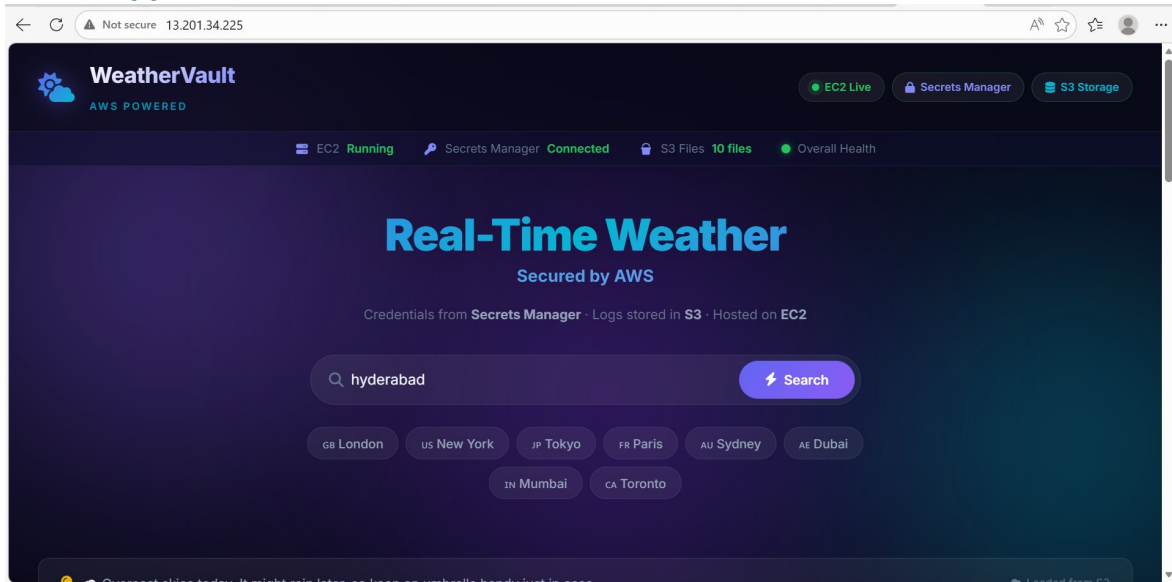
Method	Endpoint	Description	Data Source
GET	/api/health	EC2, Secrets Manager & S3 health status	All services
GET	/api/weather/current?city=London	Current weather — temperature, humidity, wind	Secrets Mgr → OWM → S3 log
GET	/api/weather/forecast?city=London	5-step weather forecast	Secrets Mgr → OWM
GET	/api/cities	Featured cities list	S3 (app-config/)
GET	/api/weather-tip?condition=rainy	Contextual weather tip	S3 (app-config/)
GET	/api/s3/info	Bucket stats & folder breakdown	S3
GET	/api/s3/weather-history	All recent search log entries	S3 (logs/)
GET	/api/s3/files?prefix=	List files matching a prefix filter	S3
GET	/api/secrets/info	Secret metadata — never exposes values	Secrets Manager
POST	/api/cache/clear	Clear in-memory Secrets + S3 caches	Internal
GET	/api/app-info	Application metadata	S3 (app-config/)

07

Screenshots & Verification

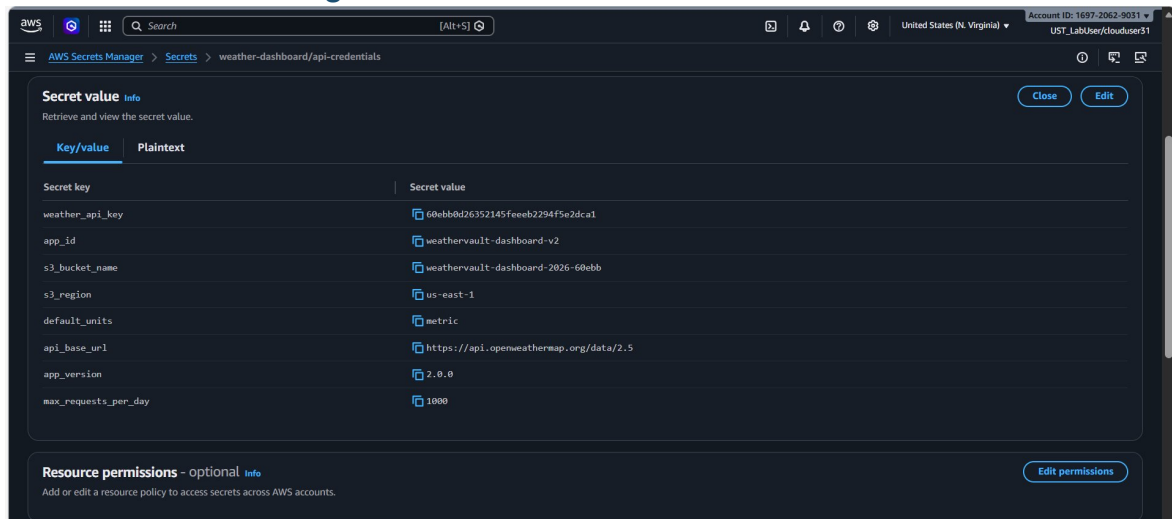
The following screenshots confirm successful deployment of each component.

7.1 Live Application — WeatherVault UI



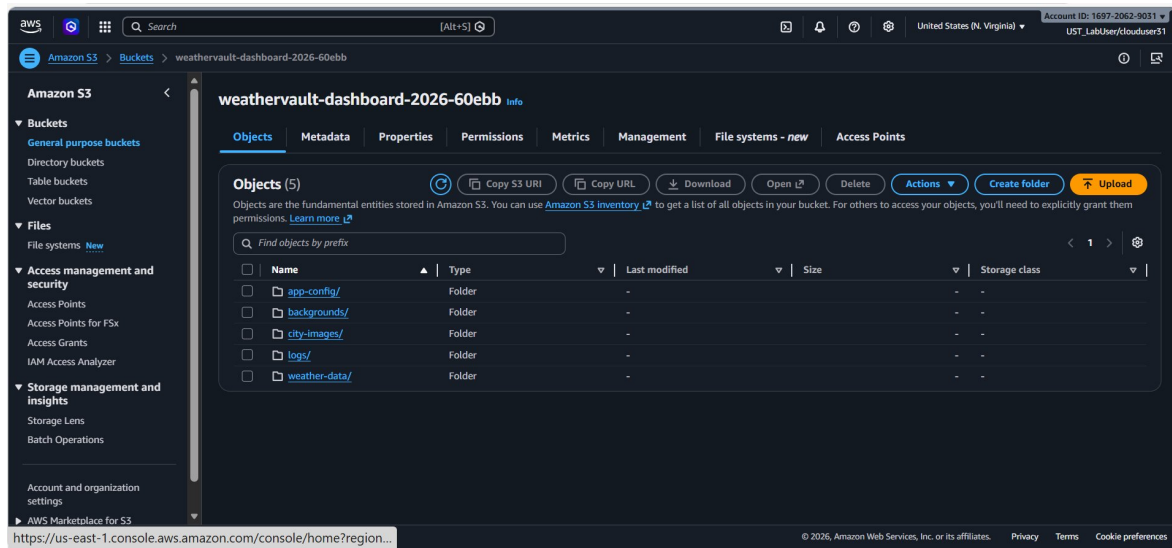
WeatherVault running at EC2 public IP 13.201.34.225 — EC2 Live, Secrets Manager Connected, S3 showing 10 files. City search and quick-access buttons are functional.

7.2 AWS Secrets Manager — Stored Credentials



Secret 'weather-dashboard/api-credentials' in us-east-1 with 8 key-value pairs including the API key, bucket name, region, and app metadata.

7.3 Amazon S3 — Bucket Structure



Bucket 'weathervault-dashboard-2026-60ebb' with 5 organised folders: *app-config*, *backgrounds*, *city-images*, *logs*, *weather-data*.